

Homework 07: Capabilities, Conflicts and a Fragile Base

Three parts. Design with capabilities, resolve a conflict Java refuses to resolve for you, then reproduce the bug that makes "prefer composition" more than a slogan.

Part One: Smart Devices

Create `Devices.java` in the classic form.

Define two interfaces:

- `Switchable` — has a `name()` the implementer must supply, plus `turnOn()` and `turnOff()` as **default** methods
- `Dimmable` — has `setLevel(int)` and `full()` as defaults, and uses a **private interface method** to clamp the level into 0..100 so the rule is written once

Then three device classes:

Device	Implements
<code>Lamp</code>	both
<code>Thermostat</code>	<code>Switchable</code> only
<code>Speaker</code>	both, but overrides <code>setLevel</code>

In `main`, loop over the devices **through the interface type**, never through the concrete class. Show that passing a `Thermostat` where a `Dimmable` is expected is impossible, and say in a comment how you know.

Part Two: A Conflict You Must Resolve

Define `Timestamped` and `Versioned`, each with a **default** method of the same name and signature. Write a class implementing both.

It will not compile. Read the error, then resolve it explicitly.

In a comment, quote the error you got and explain why Java refuses to choose for you when Python's method resolution order would have picked silently.

Part Three: Break A Subclass

Write `LoggingHashSet` extends `HashSet<String>` that keeps a log of everything added, overriding both `add` and `addAll`.

Add three items using `addAll` and print the log size. It will not be 3.

Then write `LoggingCollection` that **holds** a `HashSet` instead of extending one, with the same two methods. Add the same three items and print the log size.

Explain It

Write a comment, in your own words, covering:

1. The mechanism. Why does the inherited version over-count?
2. Why nothing in `LoggingHashSet` 's own source looks wrong.
3. What the composed version does differently that removes the problem.

Acceptance Criteria

- ☐ Runs with `java Devices.java`
- ☐ `Switchable` and `Dimmable` each have at least one `default` method
- ☐ `Dimmable` has a `private` interface method used by its defaults
- ☐ `Speaker` overrides a default, `Lamp` does not
- ☐ The loops in `main` are typed to the interface, not the class
- ☐ The conflicting-defaults class resolves with `Interface.super.method()`
- ☐ A comment quotes the real compiler error you saw
- ☐ `LoggingHashSet` prints a log size that is not 3
- ☐ `LoggingCollection` prints 3
- ☐ Your explanation names the mechanism, not just the symptom

Stretch

Change `LoggingHashSet` to extend `ArrayList<String>` instead, leaving everything else identical. The count will come out right.

Write a comment on what that tells you about the original bug, and about how much you can trust a subclass you have reviewed but whose parent you have not read.

Hint, if part three gives you 3 straight away

Check which collection you extended, and check that you added the items with `addAll` rather than three separate `add` calls. Both details matter, and one of them is the entire point of the exercise.